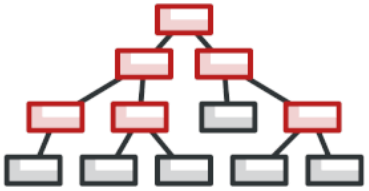
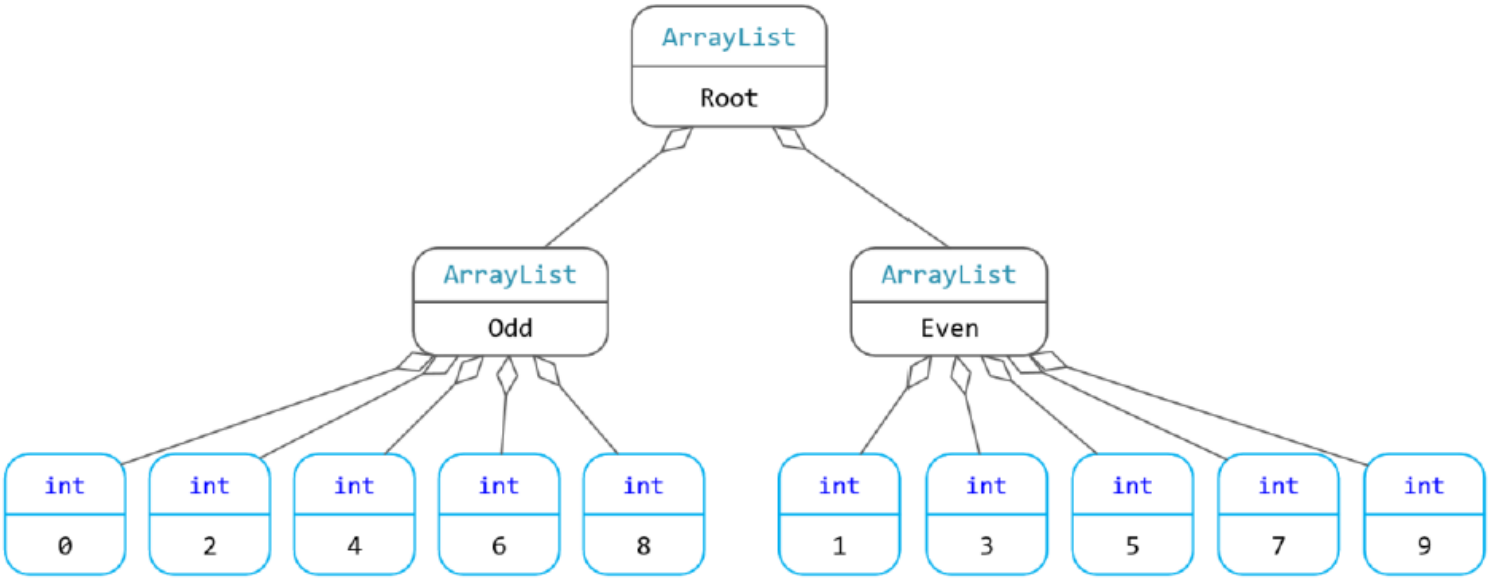
Структурные паттерны. Компоновщик



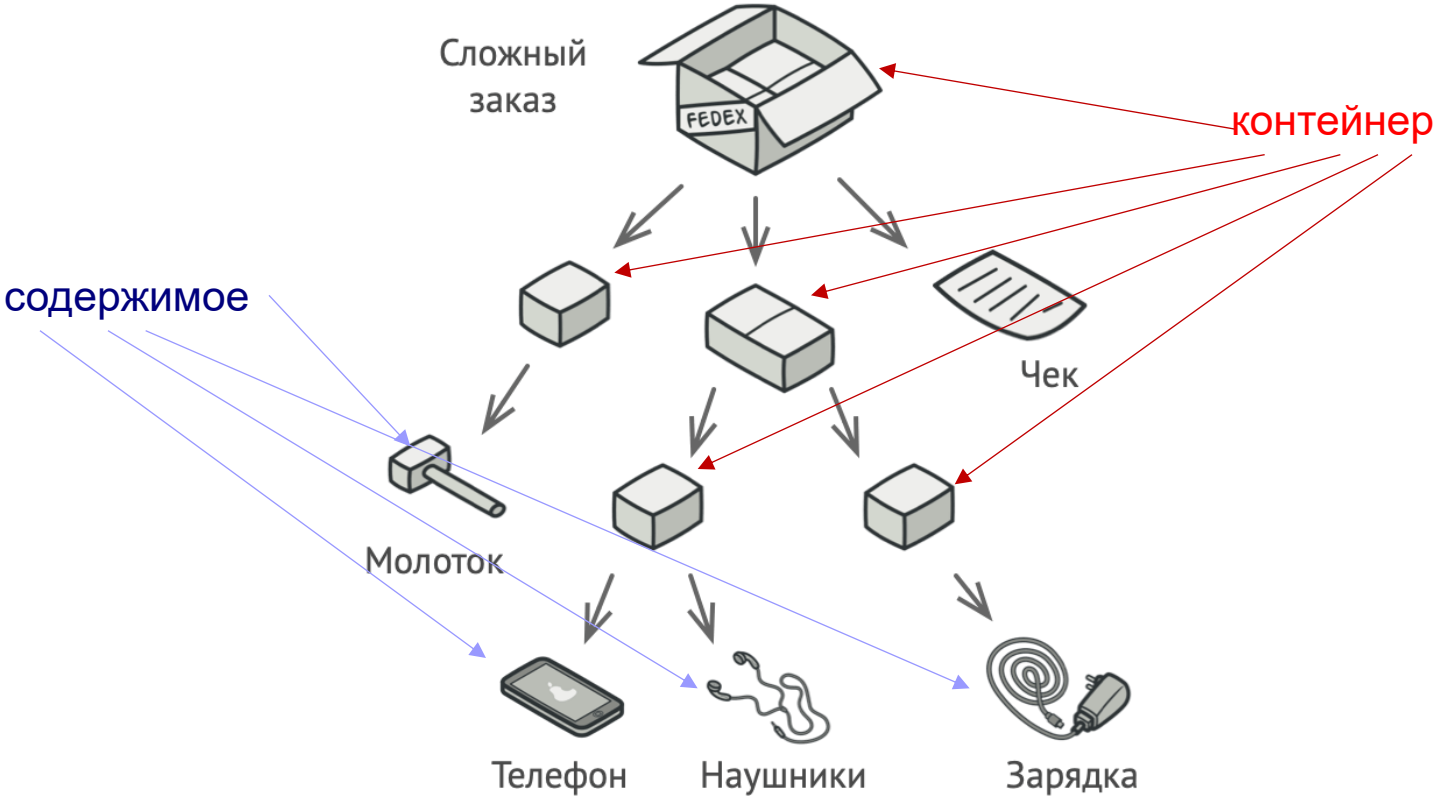
Паттерн Компоновщик
Composite



Компоновщик — это структурный паттерн проектирования, который позволяет сгруппировать множество объектов в *древовидную* структуру, а затем работать с ней так, как будто это единичный объект.

Позволяет клиентам единообразно трактовать индивидуальные объекты (***листья***) и составные объекты (***ветки***).

Проблема

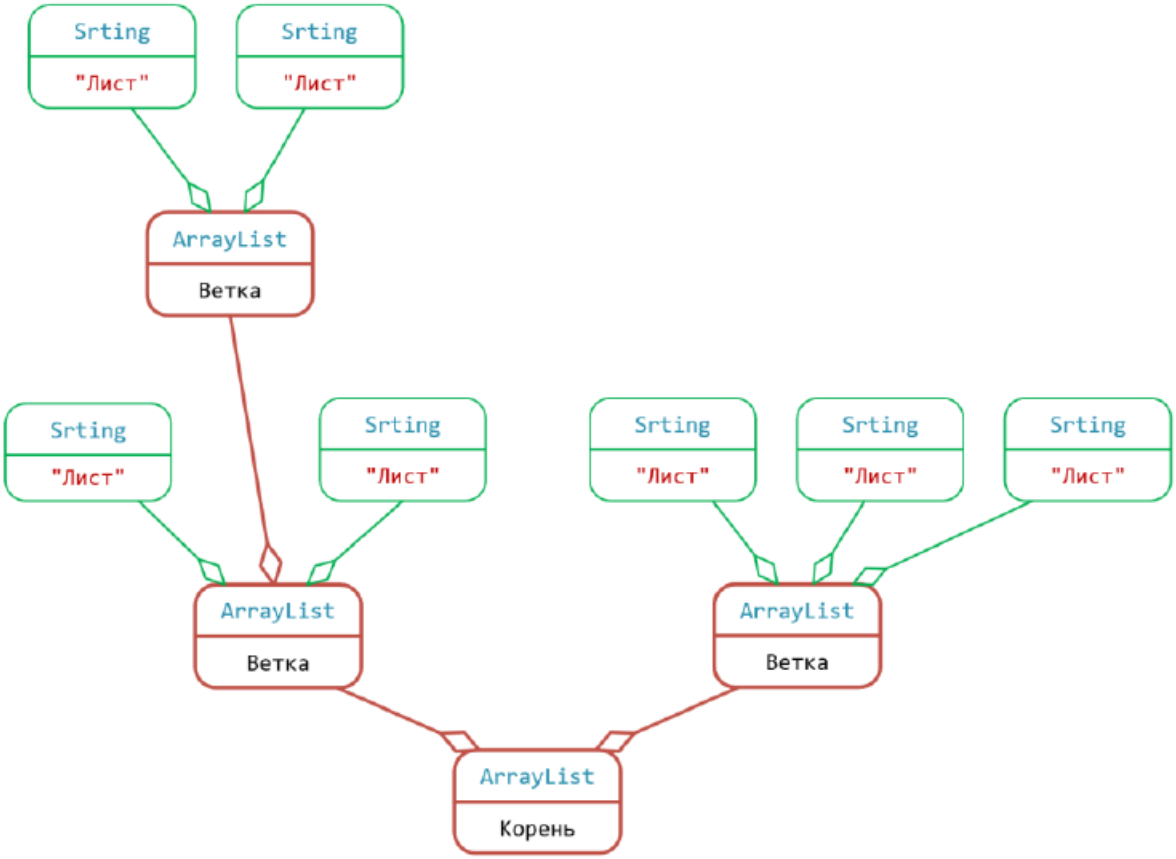


Доступ к параметрам вложенных объектов, часть из которых может быть Контейнерами для других объектов:

- типы *Контейнеров* и их *Содержимое* заранее могут быть неизвестны
- заранее неизвестно количество уровней вложенности контейнеров

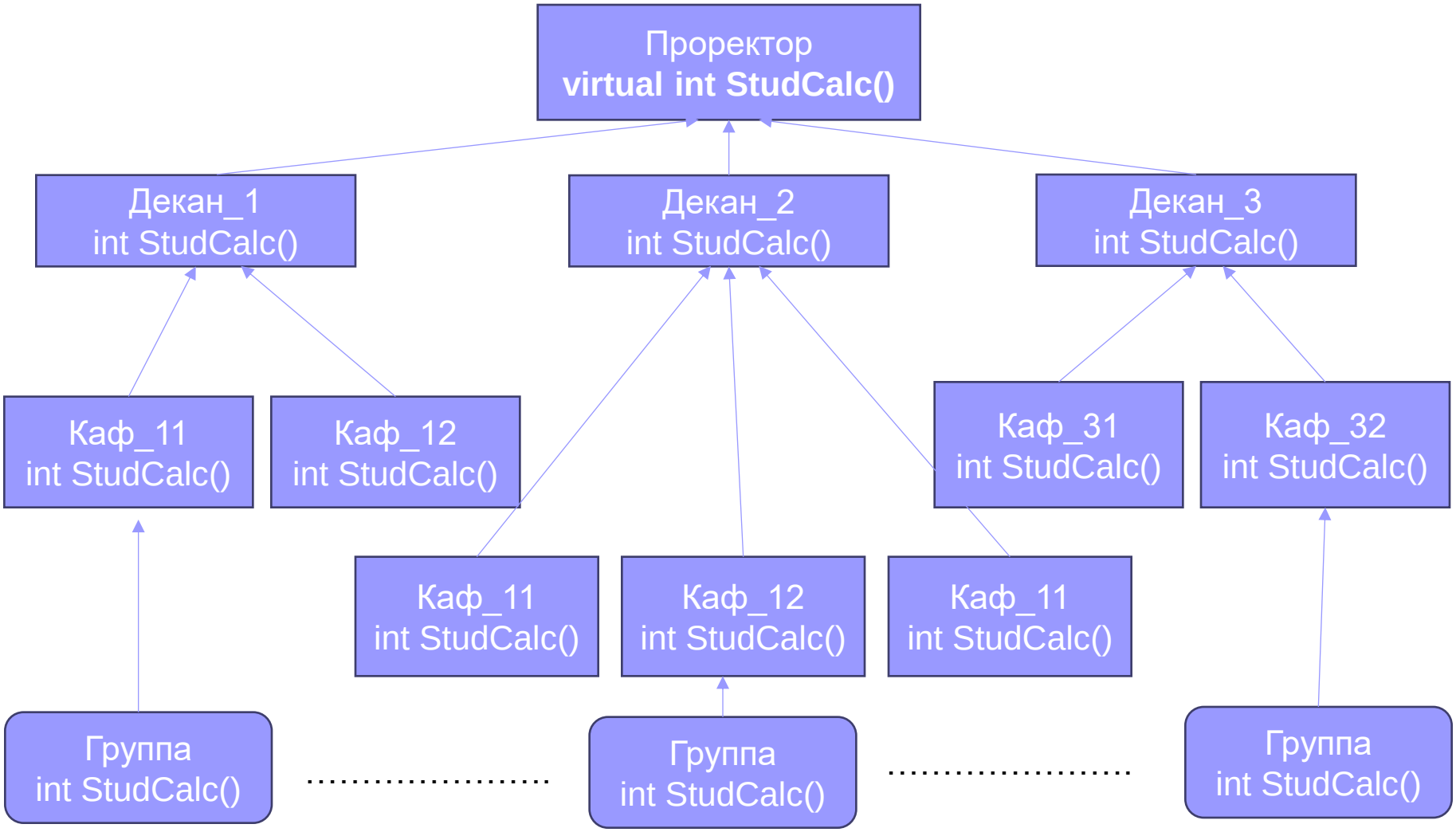
Решение

Компоновщик предлагает рассматривать *Контейнер* и *Содержимое* через единый интерфейс с общими методами.



Чтобы построить простейшее дерево, можно воспользоваться двумя классами – *ArrayList* и *String*. Корень и ветки дерева можно представить экземплярами класса (коллекции) *ArrayList*, а листья экземплярами класса *String*. Из корня и веток могут расти как листья, так и другие ветки (аналогично живому дереву). Из листьев ничего произрастать не может.

Иерархическая структура организации

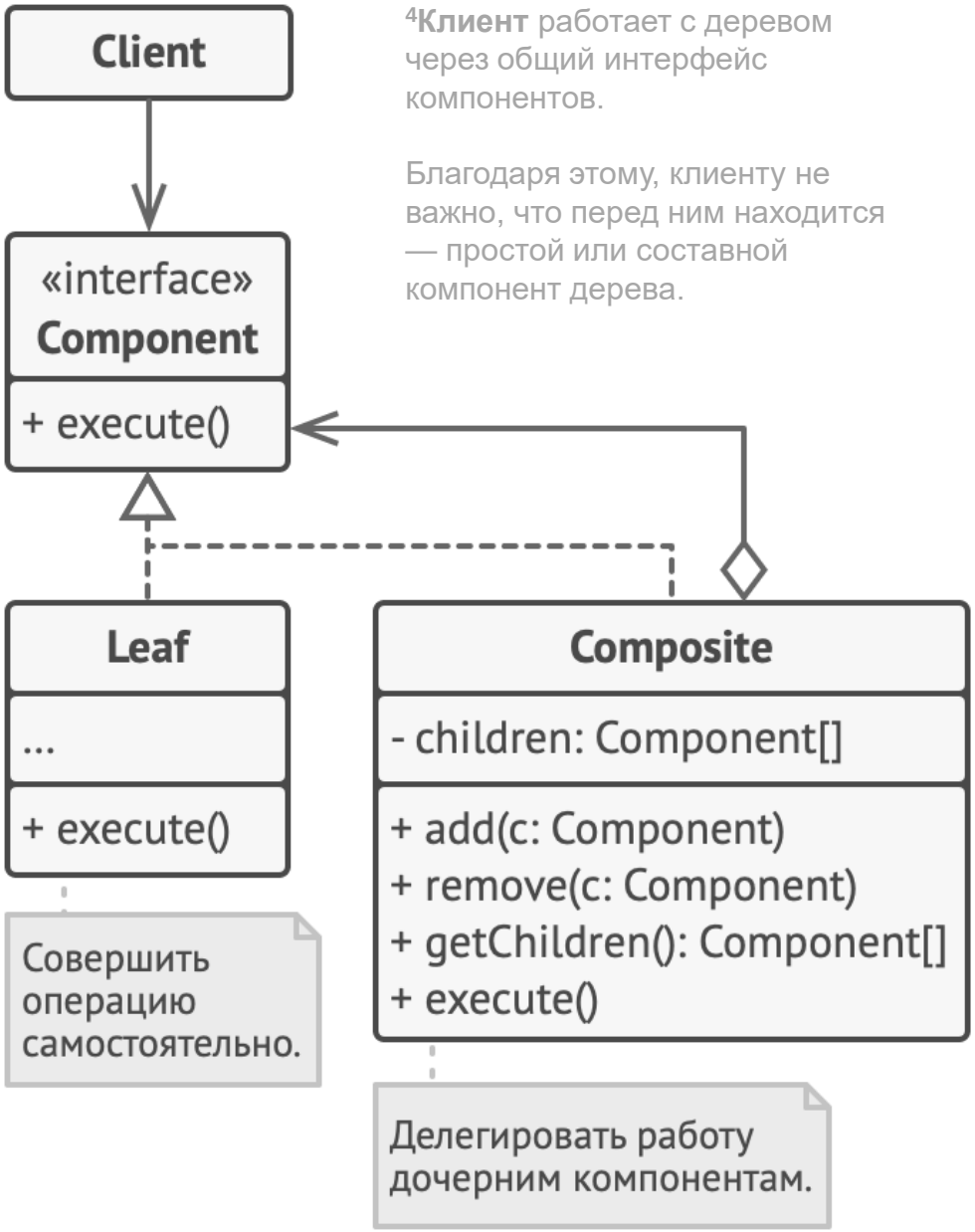


Вложенные структуры отрабатывают вызов метода **StudCalc(),** возвращая результат по иерархии

Структура паттерна

[1]**Компонент** определяет общий интерфейс для простых и составных компонентов дерева.

[2]**Лист** — это простой компонент дерева, не имеющий ответвлений. Из-за того, что им некому больше передавать выполнение, классы листьев будут содержать большую часть полезного кода.



[4]**Клиент** работает с деревом через общий интерфейс компонентов.

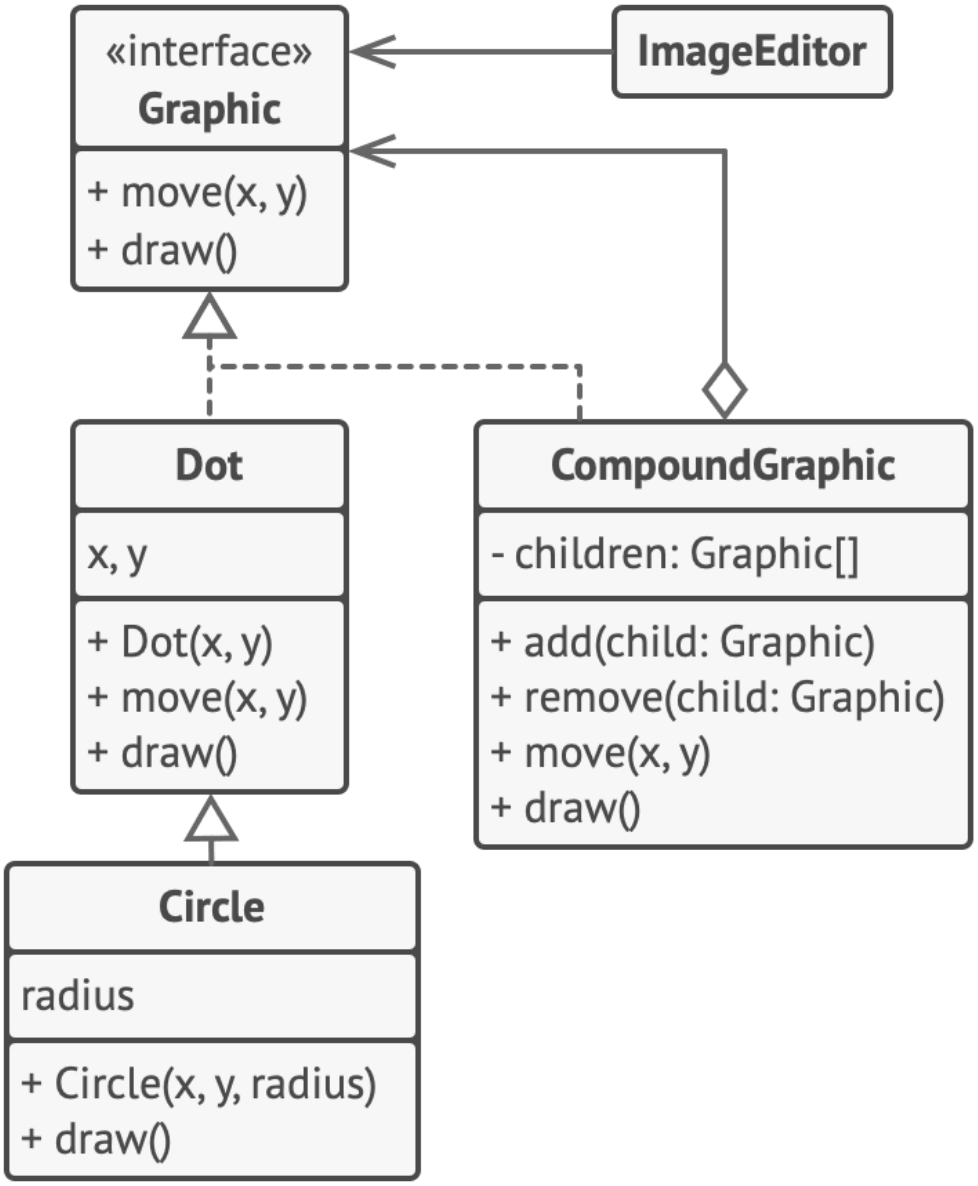
Благодаря этому, клиенту не важно, что перед ним находится — простой или составной компонент дерева.

[3]**Контейнер** (или **композит**) — это составной компонент дерева. Он содержит набор дочерних компонентов, но ничего не знает об их типах. Это могут быть как простые компоненты-листья, так и другие компоненты-контейнеры. Но это не является проблемой, если все дочерние компоненты следуют единому интерфейсу.

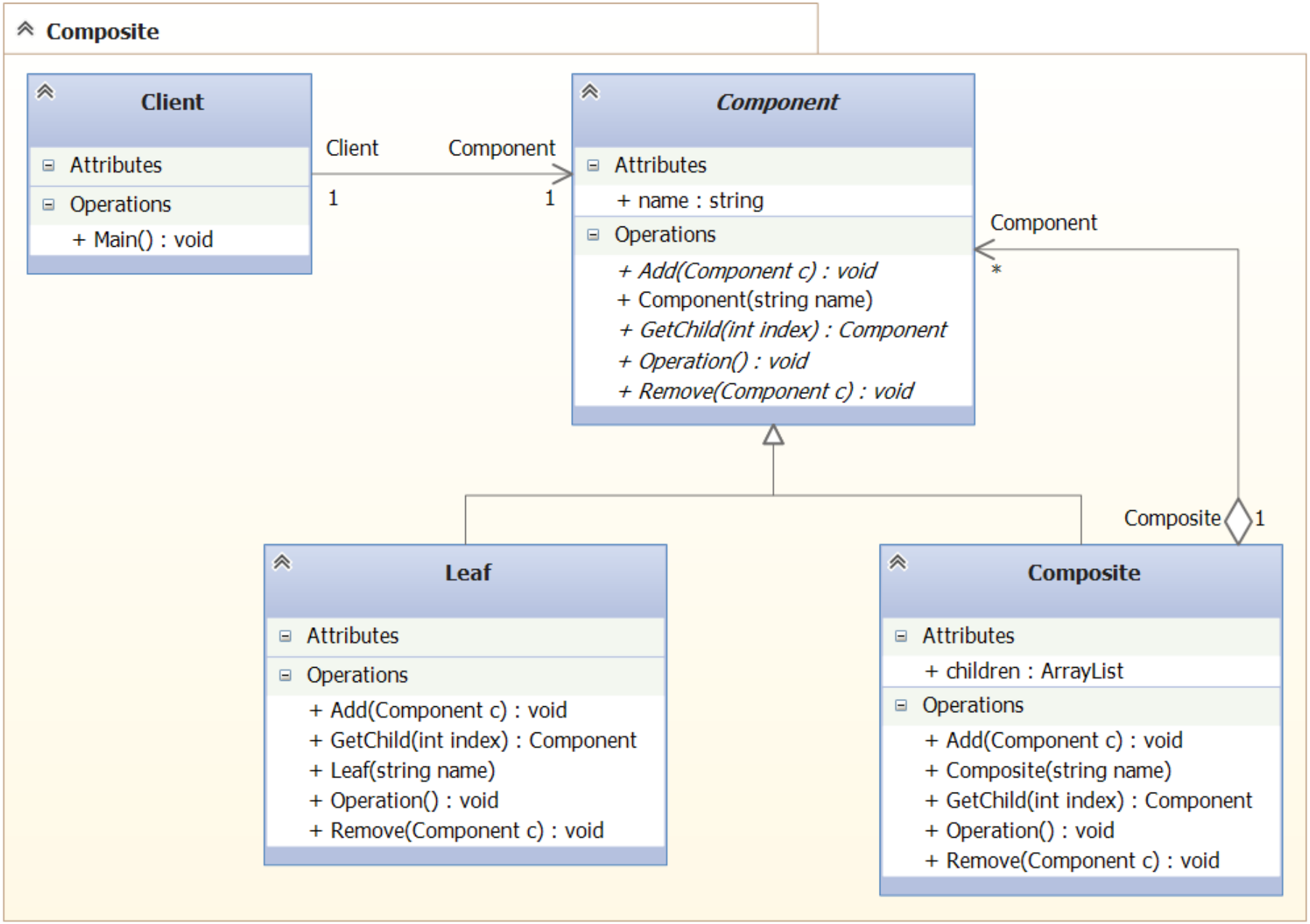
Методы контейнера переадресуют основную работу своим дочерним компонентам, хотя и могут добавлять что-то своё к результату.

Пример

Пример редактора геометрических фигур.



Структура паттерна UML



Структура паттерна на языке C#

```
class Program
{
    static void Main()
    {
        Component root = new Composite("ROOT");
        Component branch1 = new Composite("BR1");
        Component branch2 = new Composite("BR2");
        Component leaf1 = new Leaf("L1");
        Component leaf2 = new Leaf("L2");

        root.Add(branch1);
        root.Add(branch2);
        branch1.Add(leaf1);
        branch2.Add(leaf2);
        root.Operation();
    }
}
```

Client - **Клиент**:
Манипулирует объектами, входящими в структуру дерева, через интерфейс, предоставляемый классом Component.

Component - **Компонент**:
Предоставляет интерфейс для объектов из которых составляется дерево. В частном случае может предоставлять реализацию по умолчанию для некоторых методов.

```
abstract class Component
{
    protected string name;

    public Component(string name)
    {
        this.name = name;
    }

    public abstract void Operation();
    public abstract void Add(Component component);
    public abstract void Remove(Component component);
    public abstract Component GetChild(int index);
}
```

Leaf - **Лист**:
Является классом листовых узлов дерева и не может иметь потомков, т.е., включать в себя объекты относящиеся к структуре дерева (из листа не может вырасти ветвь или другой лист).

```
class Leaf : Component
{
    public Leaf(string name)
        : base(name)
    {
    }

    public override void Operation()
    {
        Console.WriteLine(name);
    }

    public override void Add(Component component)
    {
        throw new InvalidOperationException();
    }

    public override void Remove(Component component)
    {
        throw new InvalidOperationException();
    }

    public override Component GetChild(int index)
    {
        throw new InvalidOperationException();
    }
}
```

```
class Composite : Component
{
    ArrayList nodes = new ArrayList();

    public Composite(string name)
        : base(name)
    {
    }

    public override void Operation()
    {
        Console.WriteLine(name);

        foreach (Component component in nodes)
            component.Operation();
    }

    public override void Add(Component component)
    {
        nodes.Add(component);
    }

    public override void Remove(Component component)
    {
        nodes.Remove(component);
    }

    public override Component GetChild(int index)
    {
        return nodes[index] as Component;
    }
}
```

Composite - **Составной объект**:
Задает поведение объектов, входящих в структуру дерева, у которых есть потомки, а также сам хранит в себе компоненты дерева (объекты потомки), как узловые, так и листовые. Реализует методы интерфейса Component, относящиеся к управлению потомками.

Отношения между участниками

Отношения между классами

- Класс Leaf связан связью отношения наследования с абстрактным классом Component.
- Класс Composite связан связью отношения наследования и связью отношения агрегации с абстрактным классом Component.

Отношения между объектами

- Клиенты (Client) взаимодействуют с элементами, входящими в состав дерева, через интерфейс (набор методов) предоставленный абстрактным классом Component.
- Если метод вызывается на листовом объекте (Leaf), то объект Leaf и обрабатывает поступивший запрос (т.е. выполняет функциональность, заложенную в тело его метода).
- Если метод вызывается на составном (узловом) объекте (Composite), то объект Composite перенаправляет запрос (вызывает метод) своим потомкам, возможно при этом выполняя некоторые дополнительные действия до или после перенаправления запроса.

Применение

Когда нужно представить древовидную структуру объектов

Паттерн Компоновщик предлагает хранить в составных объектах ссылки на другие простые или составные объекты. Те, в свою очередь, тоже могут хранить свои вложенные объекты и так далее. В итоге можно построить сложную древовидную структуру данных, используя всего две основные разновидности объектов.

Когда клиенты должны единообразно трактовать простые и составные объекты.

Благодаря тому, что простые и составные объекты реализуют общий интерфейс, клиенту безразлично, с каким именно объектом ему предстоит работать.

Шаги реализации

1. **Если логику предметной области можно представить как древовидную структуру,** - следует разбить её на *простые* компоненты и *контейнеры*. Контейнеры могут содержать как простые компоненты, так и другие вложенные контейнеры.
2. **Создать общий интерфейс компонентов**, который объединит операции контейнеров и простых компонентов дерева.
3. **Создать класс компонентов-листьев**, не имеющих дальнейших ответвлений. Программа может содержать несколько таких классов.
4. **Создать класс компонентов-контейнеров** и добавить в него массив для хранения ссылок на вложенные компоненты. Этот массив должен быть способен содержать как простые, так и составные компоненты, поэтому его нужно объявить с типом интерфейса компонентов.

   Реализовать в контейнере методы интерфейса компонентов, помня о том, что контейнеры должны делегировать основную работу своим дочерним компонентам.
5. **Описать операции добавления и удаления дочерних компонентов** в класс контейнеров.

   Методы добавления/удаления дочерних компонентов можно поместить и в интерфейс компонентов. Это нарушит принцип разделения интерфейса, так как реализации методов будут пустыми в компонентах-листьях. Зато все компоненты дерева станут одинаковыми для клиента.

Преимущества и недостатки

+ Упрощает архитектуру клиента при работе со сложным деревом компонентов.
+ Облегчает добавление новых видов компонентов.
- Задает слишком общую структуру классов.